

Analysis 4: Real-Time Evaluation of the Landmark Supermodel

PhysioNet Utility Score, Alarm-Burden Analysis, and Dynamic Calibration

Abdul Ahad. Student ID: G00486649

11 July 2026

Table of Contents

0.1	Overview	1
0.2	Load Libraries and Set Seed	1
0.3	Locate and Read Data	1
0.4	Design Matrix Builder	2
0.5	Train the Supermodel	2
0.6	Score Every Holdout Hour	2
0.7	PhysioNet/CinC-2019 Utility Score	3
0.8	Alarm-Burden Sweep	4
0.9	Alarm Trade-Off Figure	4

0.1 Overview

This script evaluates the landmark supermodel as a streaming early-warning system on leakage-safe holdout stays. It computes: (1) the PhysioNet/CinC-2019 utility score on an hourly risk stream, (2) an alarm-burden sweep trading off sensitivity against false alarms per patient-day, and (3) a visual summary of the alarm trade-off.

0.2 Load Libraries and Set Seed

```
suppressPackageStartupMessages({ library(glmnet); library(pROC) })
set.seed(486649)
```

0.3 Locate and Read Data

```
find_data <- function(f) {
  cands <- c(file.path("../..processed_data", f),
             file.path("processed_data", f), f)
  hit <- cands[file.exists(cands)][1]; if (is.na(hit)) stop("cannot find ", f); hit
}
OUT <- dirname(find_data("11_landmark_stack.parquet"))
FIG <- if (dir.exists("figure")) "figure" else { dir.create("figure"); "figure" }

stack <- as.data.frame(arrow::read_parquet(find_data("11_landmark_stack.parquet")))
hold_ids <- read.csv(find_data("14_realtime_holdout_ids.csv"))$stay_id
```

```
hnew <- as.data.frame(arrow::read_parquet(find_data("14_realtime_holdout_features.parquet")))

vitals <- c("hr","resp_rate","map","spo2","temp_c","gcs")
labs <- c("lactate","creatinine","wbc","pf_ratio")
slopes <- c("hr_slope6","resp_rate_slope6","map_slope6","spo2_slope6")
scores <- c("sofa_total","qsofa","sirs")
```

0.4 Design Matrix Builder

Training-set medians and scaling parameters are extracted once and applied to both training and holdout data, ensuring no holdout information leaks into imputation or standardization.

```
make_ref <- function(raw) {
  med <- sapply(c(vitals, scores, labs), function(v) median(raw[[v]], na.rm=TRUE))
  list(med = med, s_mu = mean(raw$landmark_time), s_sd = sd(raw$landmark_time))
}
make_design <- function(raw, ref) {
  df <- data.frame(row.names = seq_len(nrow(raw)))
  for (v in c(vitals, scores)) { x <- raw[[v]]; x[is.na(x)] <- ref$med[[v]]; df[[v]] <- x }
  for (v in labs) { x <- raw[[v]]; df[[paste0(v, "_msg")]] <- as.integer(is.na(x));
    x[is.na(x)] <- ref$med[[v]]; df[[v]] <- x }
  for (v in slopes) { x <- raw[[v]]; df[[paste0(v, "_msg")]] <- as.integer(is.na(x));
    x[is.na(x)] <- 0; df[[v]] <- x }
  df$age <- raw$age; df$female <- as.integer(raw$gender == "F")
  base <- names(df)
  sz <- (raw$landmark_time - ref$s_mu) / ref$s_sd
  df$s <- sz; df$s2 <- sz^2
  for (v in base) df[[paste0(v, "_xS")]] <- df[[v]] * sz
  as.matrix(df)
}
```

0.5 Train the Supermodel

The holdout stays are excluded from the landmark stack, and a cross-validated elastic-net (alpha = 0.5) logistic regression is fit on the remaining data.

```
tr <- stack[!(stack$stay_id %in% hold_ids), ]
cat(sprintf("train rows %d (%d stays); holdout %d stays, %d scored hours\n",
  nrow(tr), length(unique(tr$stay_id)), length(hold_ids), nrow(hnew)))
```

```
## train rows 188119 (44389 stays); holdout 6000 stays, 152910 scored hours
```

```
ref <- make_ref(tr)
Xtr <- make_design(tr, ref); ytr <- tr$onset_within_h
cvfit <- cv.glmnet(Xtr, ytr, family="binomial", alpha=0.5, nfolds=5, standardize=TRUE)
```

0.6 Score Every Holdout Hour

The trained supermodel scores every hour of the holdout stays, producing the continuous risk stream evaluated below.

```
Xho <- make_design(hnew, ref)
hnew$risk <- as.numeric(predict(cvfit, Xho, s="lambda.min", type="response"))
```

0.7 PhysioNet/CinC-2019 Utility Score

The normalized utility score is computed across a grid of alarm thresholds. The reward window peaks at 6 hours before onset and decays linearly in both directions. A score of 0 corresponds to a never-alarm policy; 1 corresponds to a per-hour oracle.

```
DT_EARLY <- -12; DT_OPT <- -6; DT_LATE <- 3
U_FP <- -0.05; MIN_FN <- -2; MAX_TP <- 1
# per-hour utility given predicted 0/1 and relative time tau = hour - onset
u_pred1 <- function(tau, septic) {
  ifelse(!septic, U_FP,
    ifelse(tau <= DT_EARLY, U_FP,
      ifelse(tau <= DT_OPT, (tau - DT_EARLY)/(DT_OPT - DT_EARLY) * MAX_TP,
        ifelse(tau <= DT_LATE, MAX_TP*(1 - (tau - DT_OPT)/(DT_LATE - DT_OPT)), 0))))
}
u_pred0 <- function(tau, septic) {
  ifelse(!septic, 0,
    ifelse(tau <= DT_OPT, 0,
      ifelse(tau <= DT_LATE, MIN_FN*(tau - DT_OPT)/(DT_LATE - DT_OPT), MIN_FN)))
}
septic <- !is.na(hnew$onset_hour)
tau <- hnew$hour - ifelse(septic, hnew$onset_hour, Inf)

utility_at <- function(thresh) {
  pred <- hnew$risk >= thresh
  u_model <- sum(ifelse(pred, u_pred1(tau, septic), u_pred0(tau, septic)))
  u_model
}
u_no <- sum(u_pred0(tau, septic))
u_best <- sum(pmax(u_pred1(tau, septic), u_pred0(tau, septic)))
grid <- seq(0.02, 0.6, by = 0.01)
u_raw <- sapply(grid, utility_at)
norm <- (u_raw - u_no) / (u_best - u_no)
best_i <- which.max(norm)
util_tab <- data.frame(threshold = grid,
  utility_raw = round(u_raw, 1),
  utility_norm = round(norm, 4))
write.csv(util_tab, file.path(OUT, "15_analysis4_utility.csv"), row.names=FALSE)
cat(sprintf("\nPhysioNet utility (normalized): best %.4f at threshold %.2f\n",
  norm[best_i], grid[best_i]))
```

```
##
## PhysioNet utility (normalized): best 0.2886 at threshold 0.04
cat(sprintf(" (never-alarm baseline utility_norm = 0; per-hour oracle = 1)\n"))
```

```
## (never-alarm baseline utility_norm = 0; per-hour oracle = 1)
```

0.8 Alarm-Burden Sweep

For each threshold, patient-level sensitivity (fraction of septic patients who receive a timely alarm) and the false-alarm rate per patient-day are computed. Clinicians can then choose an operating point that balances early detection against alarm fatigue.

```
sweep <- lapply(grid, function(th) {
  pred <- hnew$risk >= th
  timely <- pred & septic & (tau >= DT_EARLY) & (tau <= DT_LATE)
  caught <- tapply(timely, hnew$stay_id, any)
  is_sep_pt <- tapply(septic, hnew$stay_id, any)
  sens <- mean(caught[is_sep_pt], na.rm=TRUE)
  fa <- pred & ((!septic) | (septic & tau < DT_EARLY))
  fa_per_day <- sum(fa) / sum(!septic | (septic & tau < DT_EARLY)) * 24
  nonsep_pt <- tapply(!septic, hnew$stay_id, all)
  any_fa <- tapply(pred & !septic, hnew$stay_id, any)
  fpr_pt <- mean(any_fa[nonsep_pt], na.rm=TRUE)
  data.frame(threshold=th, sensitivity=sens, fa_per_patient_day=fa_per_day,
             frac_nonseptic_pts_alarmed=fpr_pt)
})
sweep <- do.call(rbind, sweep)
sweep[, -1] <- round(sweep[, -1], 4)
write.csv(sweep, file.path(OUT, "15_analysis4_alarm_sweep.csv"), row.names=FALSE)
for (target in c(0.90, 0.70, 0.50)) {
  i <- which.min(abs(sweep$sensitivity - target))
  cat(sprintf("  sens=%.2f @ thr %.2f -> %.2f false alarms/patient-day, %.0f%% of non-septic pat
             sweep$sensitivity[i], sweep$threshold[i],
             sweep$fa_per_patient_day[i], 100*sweep$frac_nonseptic_pts_alarmed[i]))
}

##  sens=0.90 @ thr 0.02 -> 6.51 false alarms/patient-day, 86% of non-septic patients ever alar
##  sens=0.72 @ thr 0.05 -> 2.00 false alarms/patient-day, 59% of non-septic patients ever alar
##  sens=0.51 @ thr 0.09 -> 0.70 false alarms/patient-day, 30% of non-septic patients ever alar
```

0.9 Alarm Trade-Off Figure

```
png(file.path(FIG, "15_alarm_tradeoff.png"), width=1100, height=850, res=200)
plot(sweep$fa_per_patient_day, sweep$sensitivity, type="l", col="firebrick", lwd=2,
     xlab="False alarms per patient-day", ylab="Timely detection sensitivity",
     main="Alarm burden of the real-time supermodel")
pts <- c(which.min(abs(sweep$sensitivity-0.9)), which.min(abs(sweep$sensitivity-0.7)))
points(sweep$fa_per_patient_day[pts], sweep$sensitivity[pts], pch=19)
text(sweep$fa_per_patient_day[pts], sweep$sensitivity[pts],
     sprintf("thr=%.2f", sweep$threshold[pts]), pos=4, cex=0.7)
dev.off()

## pdf
## 2
cat("\nTier 4 complete.\n")
```

##

Tier 4 complete.